

Windowed Off-Policy Expert Value-Agreement Evaluation

Source code : `crl/ppo/expert_eval.py`
 (`build_expert_state_set`, `evaluate_expert_agreement`,
 `save_state_set`, `load_state_set`, `_clip`, `_greedy_actions`)
 `crl/envs/atari.py` (`AtariTask.make_snapshot_env`)
Design source: `docs/expert_value_agreement_eval.md`
 (metric defs sec.2; design decisions D1-D6; algorithm sec.4)
Status : implementation PASSED code-verifier against the design doc.

Purpose

A cheap, discriminative OFF-POLICY diagnostic: on a past task *i*, does a policy achieve the same windowed value as that task's expert, FROM THE EXPERT'S OWN STATES? Answers "can the new model play as well as the expert?" without paying for many full-episode rollouts, and without letting easy early-game play inflate the score.

This is NOT a standalone forgetting metric. It conditions on expert-reached states and says nothing about whether the policy can REACH them on its own (the reachability gap, doc sec.6). It is reported ALONGSIDE the on-policy no-op anchor (10 full greedy rollouts), never instead of it.

Value scale (shared by both procedures)

Discount gamma, sign-clip flag clip (= `task.clip_rewards`, default true).
 `clip(r) = sign(r) in {-1,0,+1} if clip (matches training value scale)`
 = `r` otherwise
Windowed value over a length-*n* window from rewards `r_0..` :
 `V_win = sum_{j=0..n-1} gamma^j * clip(r_j)` (discounted, sign-clipped)
 `raw = sum_{j=0..n-1} r_j` (unclipped game points)
Greedy action = argmax over the policy head's logits (project greedy-eval rule).

The snapshot environment (`AtariTask.make_snapshot_env`)

A PREPROCESSING-ONLY env: base ALE (frameskip=1, full 18-action set) wrapped by `AtariPreprocessing` (frame-skip/max-pool, grayscale, resize to 84x84, uint8). It emits a SINGLE 84x84 frame -- deliberately NO `FrameStackObservation`, NO `TimeLimit`, NO reward clip, and `terminal_on_life_loss` FORCED OFF.

- * The 4-frame stack is managed by the evaluator itself (the stack lives in `FrameStackObservation`, not in the ALE core), so a restored state's stack is reconstructed exactly from the stored observation.
- * `terminal_on_life_loss` off => a restored mid-episode state is not spuriously terminated by a stale life counter.
- * rewards are RAW; the caller sign-clips to reproduce the training value scale.

Procedure 1 : BUILD-EXPERT-STATE-SET (build_expert_state_set)

Precompute ONCE per task; the result is reusable across every checkpoint and method (the dominant cost saving, design D1).

Inputs

expert : the task's expert policy π_i^* (greedy source of states).
task : AtariTask i (supplies game, gamma, frame_stack fs, clip_rewards, max_steps).
device, expert_task_id (expert's head, default 0).
grid G = (15,20,25,30,40,60) candidate horizons (agent-steps), D4.
tau = 0.5 min fraction of consequential windows to accept H.
consequential_thresh = 1.0 raw points a window must score to be kept (D3).
n_source_starts = 20 distinct no-op expert starts (source trajectories).
states_per_task = 5000 max kept states sampled per task (N_i).
seed / rng : reproducibility.

Outputs

```
ExpertStateSet {  
  game, task_id, horizon H_i, gamma, frame_stack, clip_rewards,  
  states : list[ExpertState{ ale_state, stacked_obs, v_win, raw_win,  
                             steps_to_term, category }],  
  meta : generation settings + diagnostics (consequential_frac_by_H,  
      warning, n_candidates_kept, traj_lengths, category_counts) }  
Serialized via save_state_set (pickle); ale_state is picklable in ale_py.
```

Setup

```
gamma, fs, clip <- task.fields.  
max_gen <- task.max_steps if >0 else 10_000 (per-trajectory cap).  
rng <- provided or default_rng(seed).
```

Step A -- roll the expert, snapshot every visited state

```
trajectories <- empty  
for k = 0 .. n_source_starts-1:  
  env <- task.make_snapshot_env(clip_rewards=False) # raw, single frame  
  frame, _ <- env.reset(seed = seed + k) # noop_max => start diversity  
  ale <- env.unwraped.ale  
  stack <- deque([frame]*fs, maxlen=fs) # FrameStack reset semantics  
  # (fs copies of first frame)  
  traj <- empty ; done <- false ; steps <- 0
```

```

while not done and steps < max_gen:
  obs <- stack as (fs,84,84) array, oldest..newest    # obs expert acts on
  a   <- greedy expert action on obs at expert_task_id
  st  <- ale.cloneState()                             # SNAPSHOT state s_t BEFORE stepping
  frame, r, term, trunc, _ <- env.step(a)
  append (st, obs, raw reward r) to traj              # obs + reward r_t
  push frame into stack
  done <- (term or trunc) ; steps <- steps + 1
close env
if traj nonempty: append traj to trajectories
rewards[i] <- list of raw r_t for trajectory i

```

REMARK (D6): cloneState is $O(1)$. It replaces seed+action-prefix replay, which for a near-terminal state would cost an entire episode. restoreState (proc 2) is likewise $O(1)$. The AtariPreprocessing max-pool buffer needs no snapshot -- each agent step fully overwrites it.

Step B -- adaptive horizon H_i (design D4)

```

window(i, t, H):                                # expert window from step t
  n   <- min(H, len(rewards[i]) - t)           # cap at steps-to-terminal
  raw <- sum rewards[i][t .. t+n-1]
  v   <- sum_j gamma^j * clip(rewards[i][t+j]), j=0..n-1
  return (v, raw)

consequential_frac(H):                          # over ALL (trajectory, t) starts
  count starts whose window raw >= consequential_thresh, divide by total.
fracs <- { H : consequential_frac(H) for H in G }
H_i   <- smallest H in sorted(G) with fracs[H] >= tau      (D3+D4 filter)
      else max(G) and record a warning in meta.

```

REMARK (D3): sparse Atari games have mostly reward-free windows, so a naive average is dominated by vacuous $0 \approx 0$ "perfect agreement". Adapting H toward windows that actually score avoids this vacuity.

Step C -- collect consequential candidates at H_i , then sample

```

cands <- empty
for each trajectory i (length T) and each start t in 0..T-1:
  (v, raw) <- window(i, t, H_i)
  if raw < consequential_thresh: skip                # KEEP only consequential
  stt <- T - t                                         # steps to terminal
  cat <- "near_terminal" if stt <= H_i
      else "early"   if t < 20                        # first 20 agent-steps
      else "mid"
  append ExpertState(ale_state = traj[t].st,

```

```

        stacked_obs = traj[t].obs,
        v_win = v, raw_win = raw,
        steps_to_term = stt, category = cat)
if |cands| > states_per_task:
    keep a uniform random subset of size states_per_task (rng, no replacement)
H_i, gamma, fs, clip and cands are packed into the ExpertStateSet.

```

NOTE: v_win stored per state is the DISCOUNTED CLIPPED expert value V^*_{win} (primary); raw_win is the unclipped score (secondary/reporting).

Procedure 2 : EVALUATE-EXPERT-AGREEMENT (evaluate_expert_ag

Run PER policy checkpoint, PER task. Restores each expert state 0(1) and rolls the POLICY greedily on the same window, then reports the one-sided shortfall.

Inputs

```

policy, task, state_set (from proc 1), device,
task_id : the policy head to evaluate on this task,
n_envs   = 8 : snapshot envs run as a synchronous bank (vectorization).

```

Outputs (a dict)

```

agreement_gap          : mean_s [V*_win(s) - Vpi_win(s)]_+          (raw units)
relative_gap           : mean_s [ . ]_+ / V*_win(s)                 (in [0,1])
n_states, horizon (H_i)
relative_gap_by_category : {early, mid, near_terminal} -> mean relative gap
                        (NaN if a category has no kept states)
Lower is better. 0 => policy matches or beats the expert on every kept state.

```

Setup

```

H, gamma, fs, clip <- state_set fields (the SAME scale used to build v_win).
if state_set.states empty: return NaN gaps, n_states = 0.
envs <- n_envs snapshot envs (clip_rewards=False) ; ales <- their ALE cores.
reset each env once (seed=0) to initialize ALE before any restore.

```

Main loop -- process kept states in banks of n_envs

```

for each chunk of up to B = n_envs states:
    # --- restore each state and seed its frame stack ---
    for lane j, expert state es in chunk:
        ales[j].restoreState(es.ale_state)          # 0(1) emulator restore
        try: envs[j].lives <- ales[j].lives()        # sync stale life counter
        stacks[j] <- es.stacked_obs                  # rebuild the 4-frame stack

```

```

vpi[j] <- 0 ; gpow[j] <- 1 ; done[j] <- false    # per-lane accumulators

# --- roll the POLICY greedily for up to H steps (cap at terminal) ---
repeat H times (break early if all lanes done):
  acts <- batched greedy policy actions on stacks at task_id
  for each active lane j:
    frame, r, term, trunc, _ <- envs[j].step(acts[j])
    vpi[j] <- vpi[j] + gpow[j] * clip(r)          # SAME discount+clip scale
    gpow[j] <- gpow[j] * gamma
    stacks[j] <- drop oldest frame, append new frame  # slide the stack
    if term or trunc: done[j] <- true              # cap at terminal

# --- one-sided shortfall per state (design D5) ---
for lane j, expert state es in chunk:
  gap <- max(0, es.v_win - vpi[j])                # [V*_win - Vpi_win]_+
  rel <- gap / es.v_win if es.v_win > 0 else 0    # safe (D3 filter)
  record gap, rel, and rel under es.category
close all envs (finally).

return mean(gap), mean(rel), n_states, H, per-category mean(rel).

```

REMARK (D5, one-sided): the expert is the ceiling; beating it must not be penalised, so the shortfall is hinged at 0. This mirrors the training hinge $[V_k^L - V_k^G]_+$ (objective eq 26) -- eval and objective share "shortfall".

REMARK (D1, off-policy): states are the EXPERT's occupancy, not the policy's, which is why the set is reusable across checkpoints -- and why a good score here does NOT prove on-policy competence. Report with the no-op anchor.

Faithfulness notes (code quirks preserved above)

- * cloneState is taken BEFORE env.step, so a stored state s_t pairs with the obs the expert acted on and the reward r_t that action produced.
- * Frame-stack reset uses fs COPIES of the first frame (matches FrameStackObservation), and eval rebuilds the stack by sliding one frame per policy step -- the stack is never snapshotted through the ALE core.
- * envs[j].lives is synced from ales[j].lives() after restore (best-effort, wrapped in try/except) so the preprocessing wrapper's stale life counter does not corrupt subsequent life-loss bookkeeping.
- * The consequential filter and adaptive H_i together avoid the $0 \approx 0$ vacuity on sparse games; without them the relative gap would trend to 0 trivially.